第 14 讲：回溯——在选择树中尝试、撤销与继续

0. 本讲的核心问题

有些题要求找出所有可能方案：

- 所有子集；
- 所有排列；
- 所有满足条件的组合；
- 棋盘上的合法放置方式；
- 单词能否沿网格路径拼出。

暴力枚举的思路可能并没有错，但需要一种系统方法保证：

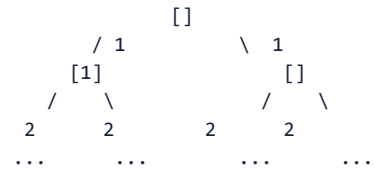
1. 每个候选方案都被考虑；
2. 不会遗漏或无意义重复；
3. 发现当前路径不可能成功时尽早停止；
4. 尝试完一个选择后，状态能恢复，再尝试其他选择。

回溯就是：

> 在决策树中进行深度优先搜索；做出选择，递归探索，返回后撤销选择。

1. 决策树视角

以 `nums = [1, 2, 3]` 的子集为例。对每个数都有两个选择：选或不选。



从根到叶子的一条路径代表一个完整决定。

回溯代码通常维护：

```
path
choices
end condition
```

2. 回溯的通用骨架

```python
result = []
path = []

def backtrack(state):
    if is_complete(state):
        result.append(path.copy())
        return

    for choice in available_choices(state):
        make_choice(choice)
        backtrack(next_state)
        undo_choice(choice)
```

最关键的三步：

撤销后，程序回到做选择之前的状态，才能正确尝试下一个分支。

---

3. 为什么必须复制 path

错误写法：

```python
result.append(path)
```

`path` 是同一个可变列表对象。后续 `append` 和 `pop` 会修改它，`result` 中此前保存的引用也会看到变化。

正确写法：

```python
result.append(path.copy())
```

或：

```python
result.append(path[:])
```

保存的是当前内容的副本。

这与链表章节的引用语义相同：变量保存对象引用，不会自动复制对象。

---

4. 子集：每个节点都是答案

对于不重复数组，使用 `start` 表示下一次只能从哪个下标之后选择。

```python
def subsets(nums):
    result = []
    path = []

    def backtrack(start):
        result.append(path.copy())

        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1)
            path.pop()

    backtrack(0)
```

```
        return result
```

4.1 为什么进入函数就记录答案

子集可以是任意长度，包括空集。决策树中的每个节点代表一个合法子集，不需要等到固定深度。

4.2 `start` 的含义

> `start` 之前的元素已经决定是否选择；下一次只能选择下标不小于 `start` 的元素。

使用 `i + 1` 进入下一层，保证：

- 同一个元素不被重复选择；
- `[1, 2]` 与 `[2, 2]` 不会作为两个子集重复出现。

4.3 复杂度

`n` 个元素有 `2^n` 个子集。仅输出就需要指数级规模：

```
O(n × 2^n)  #      path
O(n)        #           path
```

指数复杂度不是一定写错；当题目要求输出所有组合时，答案数量本身就是指数级。

---

5. 组合：选择固定数量的元素

从 `1..n` 中选择 `k` 个数：

```
[python]
def combine(n, k):
    result = []
    path = []

    def backtrack(start):
        if len(path) == k:
            result.append(path.copy())
            return

        for value in range(start, n + 1):
            path.append(value)
            backtrack(value + 1)
            path.pop()

    backtrack(1)
    return result
```

与子集的区别：

- 子集：每个长度都合法，进入函数就记录；
- 组合：只有长度等于 `k` 才记录并返回。

---

6. 组合剪枝

假设还需要选择：

```
[python]
need = k - len(path)
```

如果从 `value` 到 `n` 剩余数字不足 `need` 个，继续循环没有意义。

最大的起始选择为：

```
n - need + 1
```

因此：

```
[python]
for value in range(start, n - need + 2):
    ...
```

剪枝不是改变答案，而是提前停止不可能完成的分支。

剪枝前必须能证明：

> 被跳过的分支一定无法产生合法答案。

---

7. 排列：每层可以选择所有未使用元素

排列中顺序不同算不同答案：

```
[1, 2]   [2, 1]
```

因此不能使用只向右的 `start`。每层要从所有未使用元素中选择。

```
[python]
def permutations(nums):
    result = []
    path = []
    used = [False] * len(nums)

    def backtrack():
        if len(path) == len(nums):
            result.append(path.copy())
            return

        for i in range(len(nums)):
            if used[i]:
                continue

            used[i] = True
            path.append(nums[i])

            backtrack()

            path.pop()
            used[i] = False

    backtrack()
    return result
```

7.1 `used` 的含义

`used[i]` 表示下标 `i` 的元素是否已经出现在当前路径中。

它不是全局永久访问标记。递归返回后必须恢复为 `False`，让其他排列分支能够使用该元素。

7.2 子集、组合、排列的区别

问题	下一层候选	何时记录
子集	当前下标之后	每个节点
固定组合	当前下标之后	长度达到 k
排列	所有未使用元素	长度达到 n

---

8. 组合总和：一个元素能否重复选择

候选数可以重复使用，求和等于 target 的组合：

```
[python]
def combination_sum(candidates, target):
    result = []
    path = []

    def backtrack(start, remaining):
        if remaining == 0:
            result.append(path.copy())
            return

        if remaining < 0:
            return

        for i in range(start, len(candidates)):
            value = candidates[i]
            path.append(value)
            backtrack(i, remaining - value)
            path.pop()

    backtrack(0, target)
    return result
```

这个写法假设候选数都是正数。若包含 `0` 或负数，`remaining` 不一定持续减小，搜索可能无法终止；必须根据题目增加使用次数限制或改用其他状态设计。

注意递归传入的是 `i`，不是 `i + 1`：

```
[python]
backtrack(i, remaining - value)
```

因为当前元素允许再次选择。

若每个元素只能使用一次，则传 `i + 1`。

做题前必须写清：

---

9. 排序与去重：含重复元素的子集

输入：

```
[1, 2, 2]
```

如果直接按普通子集搜索，会多次产生 `[1, 2]`。

先排序，让相同值相邻；同一递归层中跳过相同选择：

```
[python]
def subsets_with_duplicates(nums):
    nums.sort()
    result = []
    path = []

    def backtrack(start):
        result.append(path.copy())

        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue

            path.append(nums[i])
            backtrack(i + 1)
            path.pop()

    backtrack(0)
    return result
```

9.1 为什么条件是 `i > start`

它只跳过同一层中的重复选择。

不同层仍可选择相同值。例如输入有两个 `2` 时，子集 `[2, 2]` 应保留：

- 第一层选择第一个 `2`；
- 下一层选择第二个 `2`。

如果写成 `i > 0`，会错误禁止不同层使用重复值。

---

10. 含重复元素的全排列

同样先排序，再在同一层避免选取等价的重复元素：

```
[python]
def unique_permutations(nums):
    nums.sort()
    result = []
    path = []
    used = [False] * len(nums)

    def backtrack():
        if len(path) == len(nums):
            result.append(path.copy())
            return

        for i in range(len(nums)):
            if used[i]:
                continue

            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue

            used[i] = True
            path.append(nums[i])

            backtrack()

            path.pop()
            used[i] = False

    backtrack()
    return result
```

条件：

```
[python]
nums[i] == nums[i - 1] and not used[i - 1]
```

表示同一层中，前一个相同值还没有被使用时，不允许跳过它去选后一个相同值，从而规定一种统一顺序。

这条规则较难，不要只背。画出 `[1, 1, 2]` 的前两层决策树观察重复分支。

---

11. 括号生成

生成 `n` 对合法括号。任意前缀必须满足：

≤

```
[python]
def generate_parentheses(n):
    result = []
    path = []

    def backtrack(open_count, close_count):
        if len(path) == 2 * n:
            result.append("".join(path))
            return

        if open_count < n:
            path.append("(")
            backtrack(open_count + 1, close_count)
            path.pop()

        if close_count < open_count:
            path.append(")")
            backtrack(open_count, close_count + 1)
            path.pop()

    backtrack(0, 0)
    return result
```

这道题展示了剪枝的价值：非法前缀根本不进入搜索树。

---

12. 网格回溯：单词搜索

题目：单词能否由网格中上下左右相邻字符组成，同一格不能重复使用。

```
[python]
def word_exists(board, word):
    if not word:
        return True
    if not board or not board[0]:
        return False

    rows = len(board)
    cols = len(board[0])
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    def backtrack(row, col, index):
        if board[row][col] != word[index]:
            return False

        if index == len(word) - 1:
            return True

        original = board[row][col]
        board[row][col] = "#"
```

```
                for dr, dc in directions:
                    nr = row + dr
                    nc = col + dc

                    if 0 <= nr < rows and 0 <= nc < cols:
                        if board[nr][nc] != "#":
                            if backtrack(nr, nc, index + 1):
                                board[row][col] = original
                                return True

                board[row][col] = original
                return False

            for row in range(rows):
                for col in range(cols):
                    if backtrack(row, col, 0):
                        return True

            return False
```

12.1 状态含义

```
(row, col)
index        word[index]
board    #
```

12.2 为什么返回 True 前也要恢复

如果题目允许修改输入且找到后立即结束，技术上可能不影响返回结果。但保持“任何函数返回前恢复现场”更安全，也让函数没有意外副作用。

12.3 visited 为什么是路径级状态

某个格子不能在同一条路径中重复使用，但可以在另一条候选路径中使用。因此进入时标记，离开时撤销。

这与普通图遍历的全局 `visited` 不同。

---

13. 回溯与普通 DFS 的区别

两者都使用深度优先搜索，但访问标记的生命周期常不同。

普通图 DFS

回溯

判断依据：

> 这个状态是“全局已经处理”，还是“当前方案暂时使用”？

---

14. 返回布尔值还是收集所有答案

14.1 找到一个答案即可

```
def backtrack(...):
    if goal:
        return True

    for choice in choices:
        make(choice)

        if backtrack(...):
            undo(choice)
            return True

        undo(choice)

    return False
```

14.2 收集所有答案

```
def backtrack(...):
    if goal:
        result.append(path.copy())
        return

    for choice in choices:
        make(choice)
        backtrack(...)
        undo(choice)
```

先确定题目要一个解、判断存在，还是全部解。返回设计会不同。

---

15. 怎样把思路写成回溯代码

编码前填写：

```
1. path
2.
3.
4.
5.
6.
7.
8.
9.
10.
```

如果这些问题还没有答案，不要急着写模板。

---

16. 常见错误

16.1 忘记撤销选择

`append` 后通常要有对应的 `pop`；修改棋盘后要恢复原值；设置 `used[i] = True` 后要恢复 `False`。

16.2 把 path 本身加入结果

必须复制：

```
[python]
result.append(path.copy())
```

16.3 `start`、`i`、`i+1` 混淆

- 不允许重复选当前元素：下一层传 `i + 1`；
- 允许重复选：下一层传 `i`；
- 排列：通常不用 start，而用 used。

16.4 结束条件记录后忘记 `return`

固定长度答案形成后通常应返回，否则还会继续添加元素。

16.5 去重条件跨错层级

组合/子集常用：

```
[python]
i > start and nums[i] == nums[i - 1]
```

强调“同一递归层”。

16.6 剪枝没有正确性依据

不能因为“感觉不会成功”就跳过。剪枝必须由剩余数量、排序后的大小关系或约束条件证明。

16.7 全局 `visited` 与路径 `visited` 混淆

回溯路径中的占用状态通常需要撤销。

---

17. 复杂度怎样分析

回溯复杂度通常由决策树规模决定：

```
^
```

但不同问题的精确答案数量不同：

- 子集：`2^n` 个结果；
- 排列：`n!` 个结果；
- 固定组合：`C(n, k)` 个结果。

还要考虑复制答案的成本。

回溯优化的主要方向不是把指数复杂度变成多项式，而是：

- 不进入必然失败的分支；
- 避免生成重复答案；
- 尽早返回；
- 用约束缩小候选集合。

---

18. 本讲检查表

```
1.
2. path
3.
4.
5.      path.copy()
6.
7.
8.      i    i+1
9.      used
10.
11.
12.
```

19. 本讲总结

1. 回溯是对选择树进行 DFS：选择、递归、撤销。
2. `path` 表示当前方案，保存答案时必须复制。
3. 子集和组合用 start 控制选择顺序；排列用 used 记录当前路径占用。
4. 传 `i` 表示可以重复使用当前元素，传 `i+1` 表示不能重复使用。
5. 排序后在同一层跳过相同值，可以去除重复答案。
6. 普通 DFS 的 visited 常是全局的；回溯占用状态通常属于当前路径，离开时要恢复。
7. 剪枝必须有正确性依据，而不是凭感觉跳过。